Magical Potion Brewing

By Yao Zhao

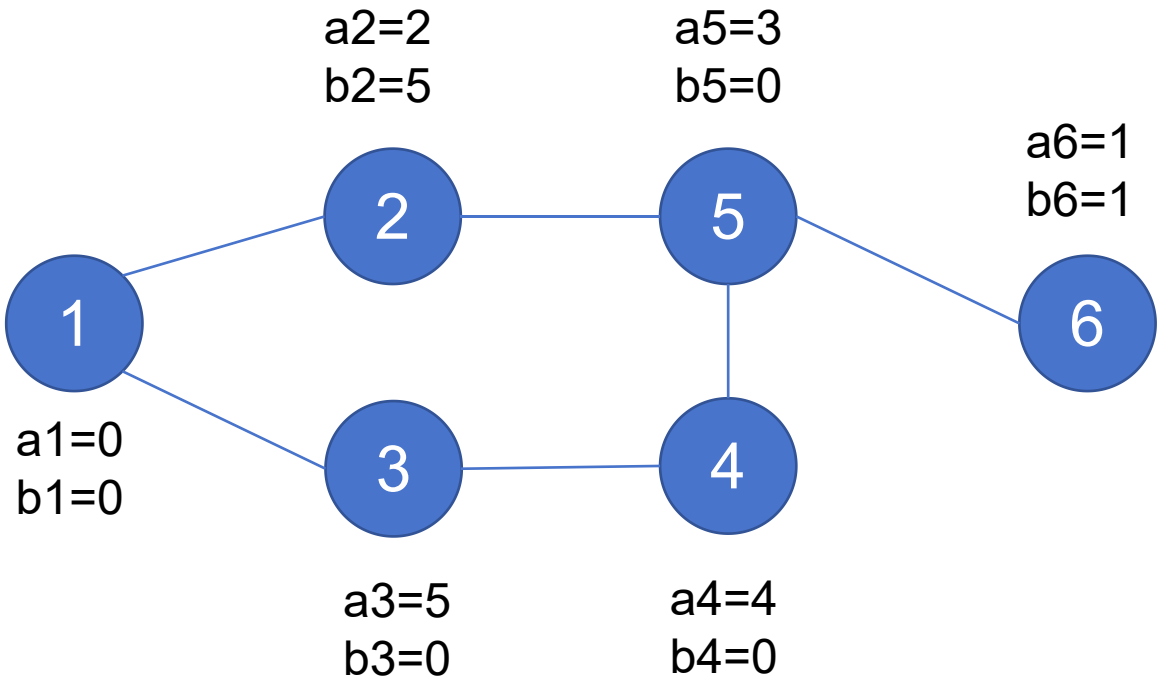
What the question about?

- You are a magic apprentice who needs to craft **Material n** step by step starting from Material **1.** During the process, you should find the **minimum total time**.
- Key Rules:
  - $\mathbf{a_i}$: The time required to craft this material itself.
  - $\mathbf{b_i}$: Once you craft this material, every subsequent crafting step will take an extra $\mathbf{b_i}$ time.

  For example: if you first craft a material with $\mathbf{b=5}$, every later step will add **5** extra time.

Sample 1

```
6 6
0 2 5 4 3 1
0 5 0 0 0 1
1 2
1 3
2 5
3 4
4 5
5 6
```



Way1: 1→2→5→6

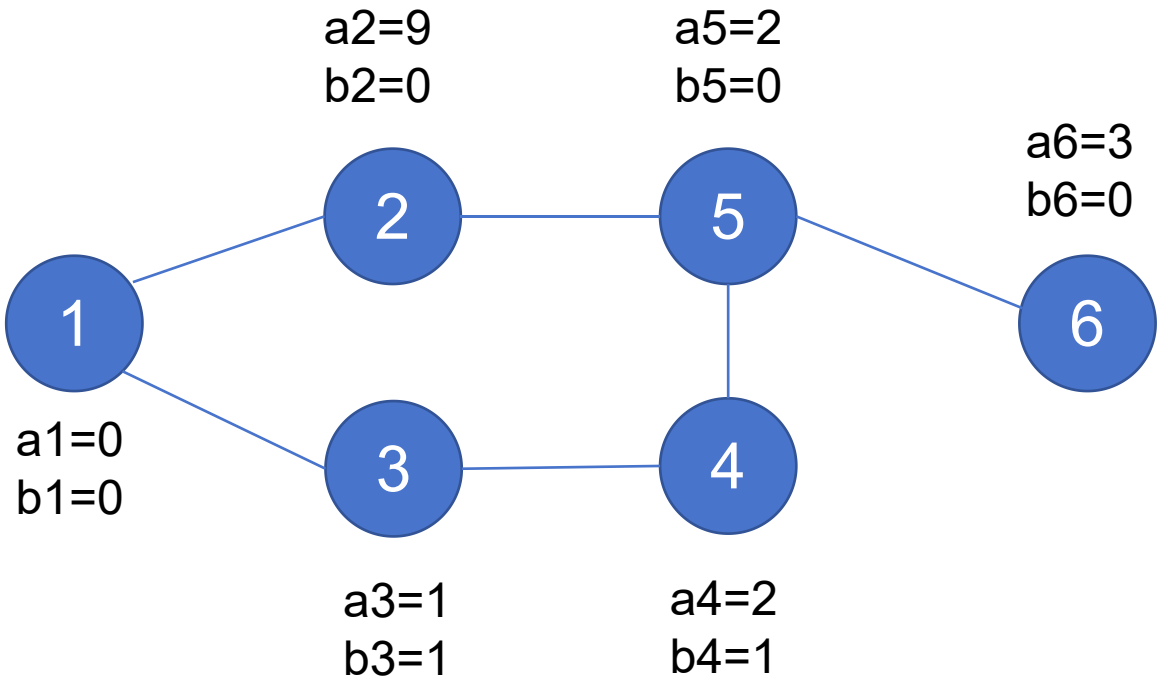
cost1
=a1+b1*3+a2+b2*2+a5+b5*1+a6=0+0*3+2+5*2+3+0*1+1=16

Way2: 1→3→4→5→6

cost2
=a1+b1*4+a3+b3*3+a4+b4*2+a5+b5*1+a6=0+0*4+5+0*3+4+0*2+3+0*1+1=13

Sample 2

```
6 6
0 9 1 2 2 3
0 0 1 1 0 0
1 2
1 3
2 5
3 4
4 5
5 6
```



Way1: 1→2→5→6

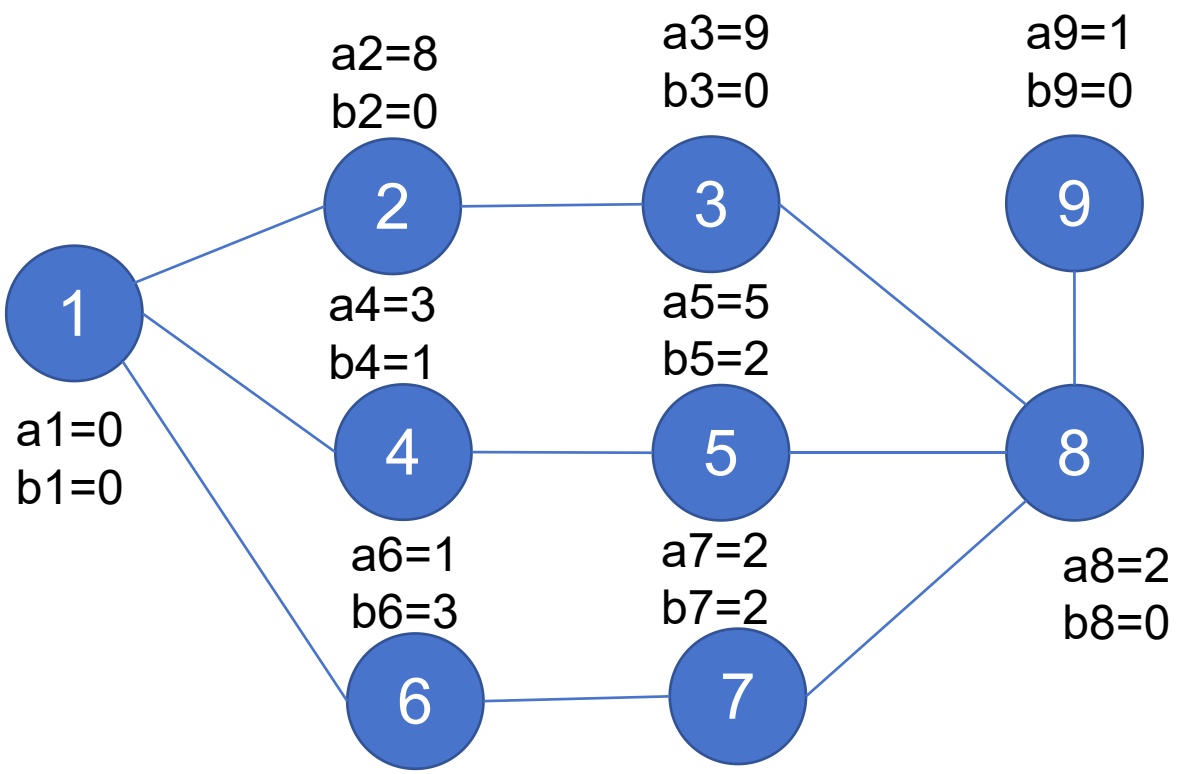
cost1
=a1+b1*3+a2+b2*2+a5+b5*1+a6=0+0*3+9+0*2+2+0*1+3=14

Way2: 1→3→4→5→6

cost2
=a1+b1*4+a3+b3*3+a4+b4*2+a5+b5*1+a6=0+0*4+1+1*3+2+1*2+2+0*1+3=13

Sample 3

```
9 10
0 8 9 3 5 1 2 2 1
0 0 0 1 2 3 2 0 0
1 2
1 4
1 6
2 3
3 8
4 5
5 8
6 7
7 8
8 9
```



Way1: 1→2→3→8→9

cost1
=a1+b1*4+a2+b2*3+a3+b3*2+a8+b8*1
+a9=0+0*4+8+0*3+9+0*2+2+0*1+1=20

Way2: 1→4→5→8→9

cost2
=a1+b1*4+a4+b4*3+a5+b5*2+a8+b8*1
+a9=0+0*4+3+1*3+5+2*2+2+0*1+1=18

Way3: 1→6→7→8→9

cost3
=a1+b1*4+a6+b6*3+a7+b7*2+a8+b8*1
+a9=0+0*4+1+3*3+2+2*2+2+0*1+1=19

Static View

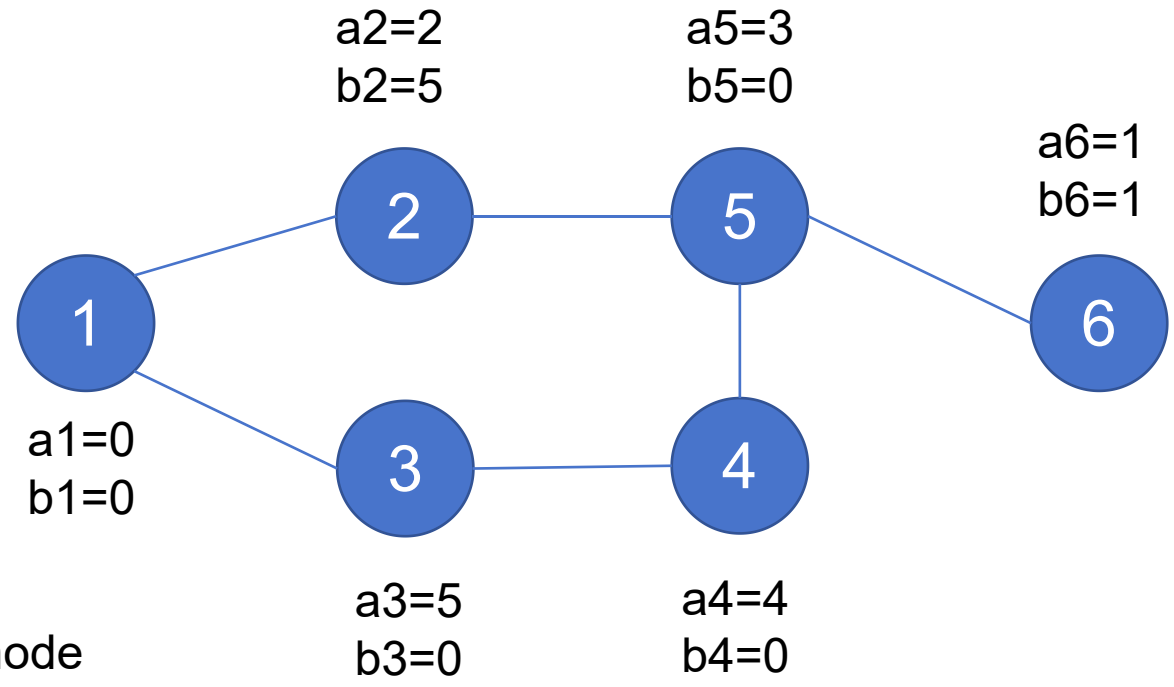
- path 1→n: Total cost = $\sum ai + \sum (bi \times (nodes\ number\ after\ node\ i))$

Shortest Path

- BFS: unweighted graphs
- **Dijkstra:non- negative weighted graphs**

Try Dijkstra: Sample 1 Walkthrough

dis[]	1	2	3	4	5	6
initial	0	∞	∞	∞	∞	∞
iter 1	~~(0)~~	(2)	(5)	(∞)	(∞)	(∞)
iter 2	~~(0)~~	~~(2)~~	(5)	(∞)	(8)	(∞)
iter 3	~~(0)~~	~~(2)~~	~~(5)~~	(9)	(8)	(∞)
iter 4	~~(0)~~	~~(2)~~	~~(5)~~	(9)	~~(8)~~	(14)
iter 5	~~(0)~~	~~(2)~~	~~(5)~~	~~(9)~~	~~(8)~~	(14)



generate (12) , but node 5 is visited and 12 > 8

In stardard dijkstra, (12) cannot enter to PQ again when node 5 is labed visited. So the path 1→3→4→5 has no chance to update the distance of node 6 to 13.

🚫 Why Standard Dijkstra Fails?

Path	Cost to reach node 5	Cumulative k	Future Impact
Path 1→2→5	8	5	each step +5
Path 1→3→4→5	12	0	each step +0

- Cost depends on path history
- Same node can have multiple valid states
- Greedy property is broken

Correct State Design

- State = (node, c, k), where c is cost so far, k is accumulated penalty(∑b on the path)
- Track both cost and history

State Transition

Assume a state (u, c, k), for each neighbor v of u:

- new_c = c + a[v] + k
- new_k = k + b[v]

new state (v, new_c, new_k)

Node index

Total cost

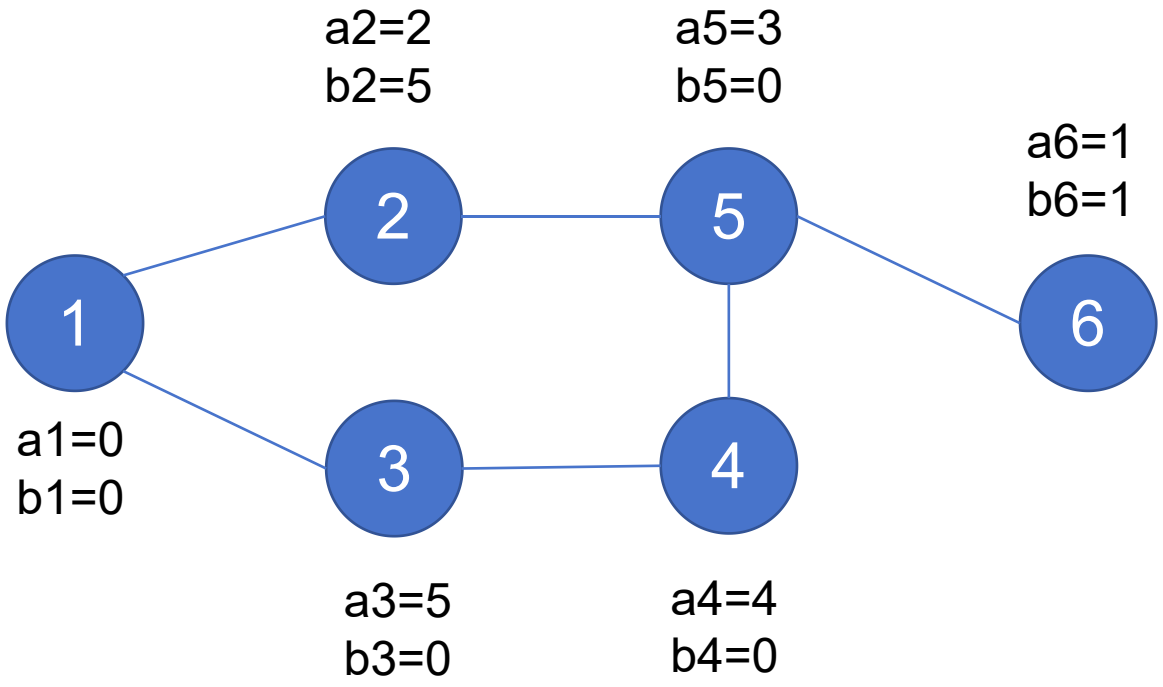
Accumulated penalty

Modified Dijkstra

- Expand states instead of nodes (PQ poll the state with the smallest **cost** first)
- First reach target = optimal

Try Modified Dijkstra: Sample 1

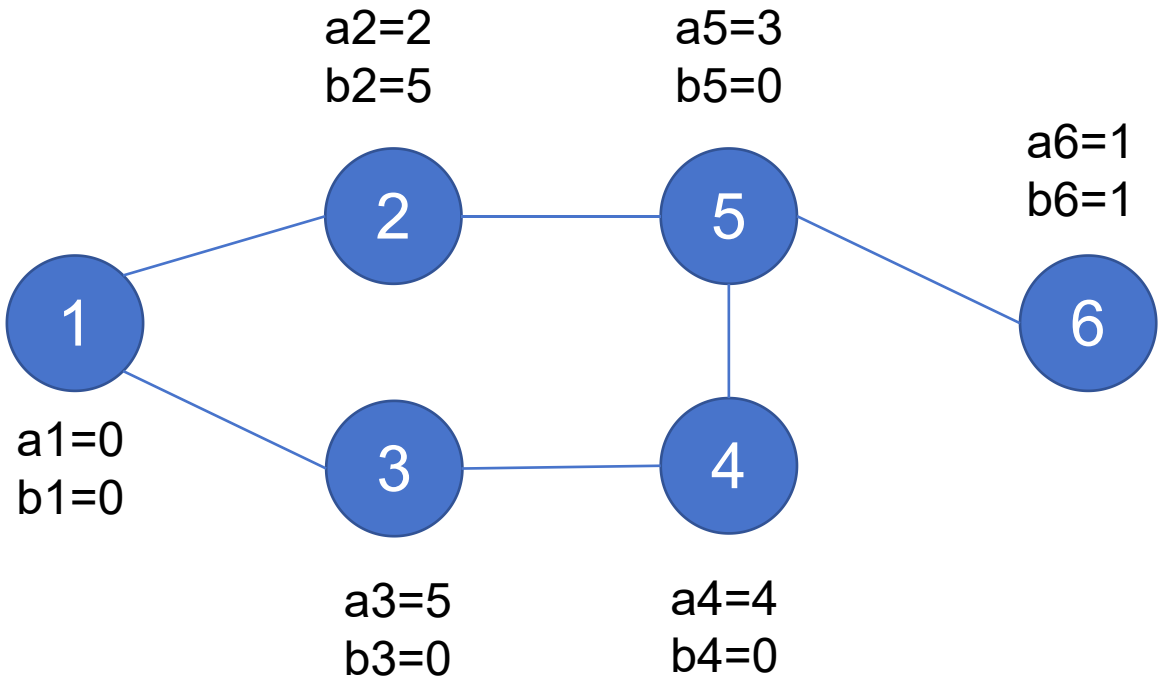
states	1	2	3	4	5	6
initial	(0, 0)	(∞, 0)	(∞, 0)	(∞, 0)	(∞, 0)	(∞, 0)
iter 1	~~(0, 0)~~	(∞, 0) (2, 5)	(∞, 0) (5, 0)	(∞, 0)	(∞, 0)	(∞, 0)
iter 2	~~(0, 0)~~ (2, 5)	(∞, 0) ~~(2, 5)~~	(∞, 0) (5, 0)	(∞, 0)	(∞, 0) (8, 5)	(∞, 0)
iter 3	~~(0, 0)~~ ~~(2, 5)~~	(∞, 0) ~~(2, 5)~~ (9,10)	(∞, 0) (5, 0) (12,5)	(∞, 0)	(∞, 0) (8, 5)	(∞, 0)
iter 4	~~(0, 0)~~ ~~(2, 5)~~ (5,0)	(∞, 0) ~~(2, 5)~~ (9,10)	(∞, 0) ~~(5, 0)~~ (12,5)	(∞, 0) (9, 0)	(∞, 0) (8, 5)	(∞, 0)
iter 5	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~	(∞, 0) ~~(2, 5)~~ (9,10) (7,5)	(∞, 0) ~~(5, 0)~~ (12,5) (17,5)	(∞, 0) (9, 0)	(∞, 0) (8, 5)	(∞, 0)



Maintain (c, k) for each u:
c: cost of path 1->u
k: cumulative b from history path 1->u
When we go to v from u, then
push v's states (v, v + a[v] + k, k + b[v]) to pq

Try Modified Dijkstra: Sample 1

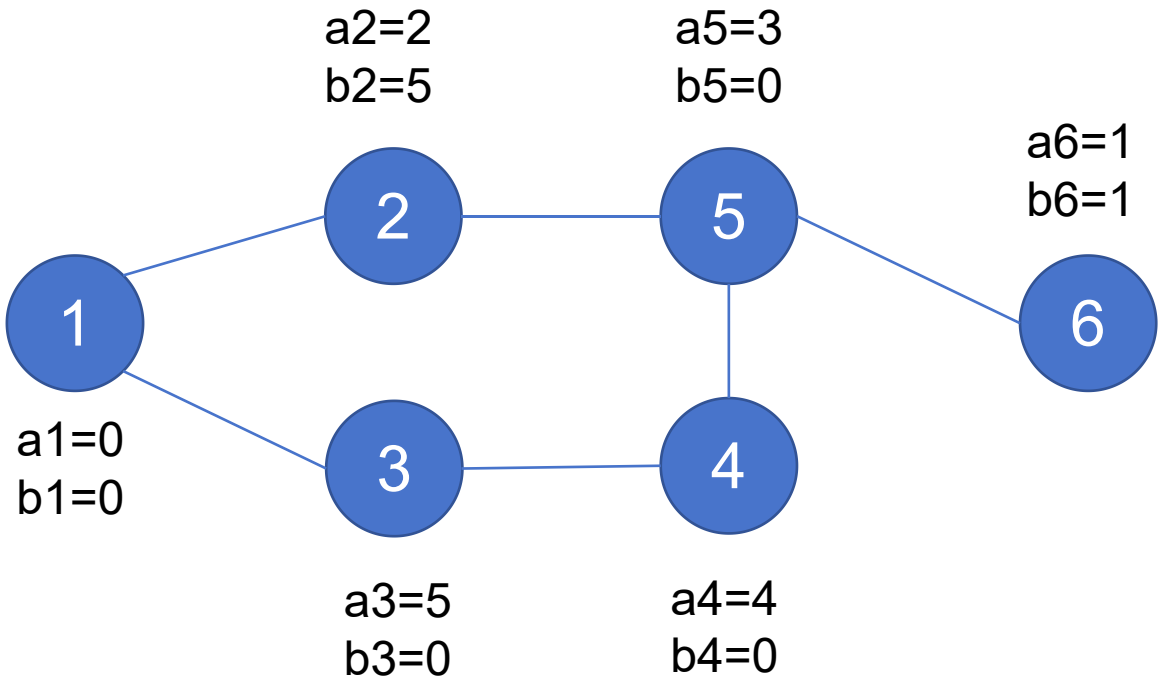
states	1	2	3	4	5	6
iter 6	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~ (...,..)	(∞, 0) ~~(2, 5)~~ (9,10) ~~(7,5)~~	(∞, 0) ~~(5, 0)~~ (12,5) (17,5)	(∞, 0) (9, 0)	(∞, 0) (8, 5) (10,5)	(∞, 0)
iter7	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~ (...,..)	(∞, 0) ~~(2, 5)~~ (9,10) ~~(7,5)~~ (15,10)	(∞, 0) ~~(5, 0)~~ (12,5) (17,5)	(∞, 0) (9, 0)	(∞, 0) ~~(8, 5)~~ (15,5)	(∞, 0) (14,6)
iter 8	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~ (...,..)	(∞, 0) ~~(2, 5)~~ ~~(9,10)~~ ~~(7,5)~~ (15,10)	(∞, 0) ~~(5, 0)~~ (12,5) (17,5) (...,...)	(∞, 0) (9, 0)	(∞, 0) ~~(8, 5)~~ (15,5)	(∞, 0) (14,6)



Maintain (c, k) for each u:
c: cost of path 1->u
k: cumulative b from history path 1->u
When we go to v from u, then
push v's states (v, v + a[v] + k, k + b[v]) to pq

Try Modified Dijkstra: Sample 1

states	1	2	3	4	5	6
iter 9	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~ (...,..)	(∞, 0) ~~(2, 5)~~ ~~(9,10)~~ ~~(7,5)~~ (15,10)	(∞, 0) ~~(5, 0)~~ (12,5) (17,5) (...,...) (...,...)	(∞, 0) ~~(9, 0)~~	(∞, 0) ~~(8, 5)~~ (15,5) (12,0)	(∞, 0) (14,6)
iter10	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~ (...,..) (...,..)	(∞, 0) ~~(2, 5)~~ (9,10) ~~(7,5)~~ (15,10)	(∞, 0) ~~(5, 0)~~ ~~(12,5)~~ (17,5)	(∞, 0) ~~(9, 0)~~ (...,...)	(∞, 0) ~~(8, 5)~~ (15,5) (12,0)	(∞, 0) (14,6)
iter 11	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~ (...,..) (...,..)	(∞, 0) ~~(2, 5)~~ (9,10) ~~(7,5)~~ (15,10) (...,...)	(∞, 0) ~~(5, 0)~~ ~~(12,5)~~ (17,5)	(∞, 0) ~~(9, 0)~~ (...,...)	(∞, 0) ~~(8, 5)~~ (15,5) ~~(12,0)~~	(∞, 0) (14,6) (13,1)

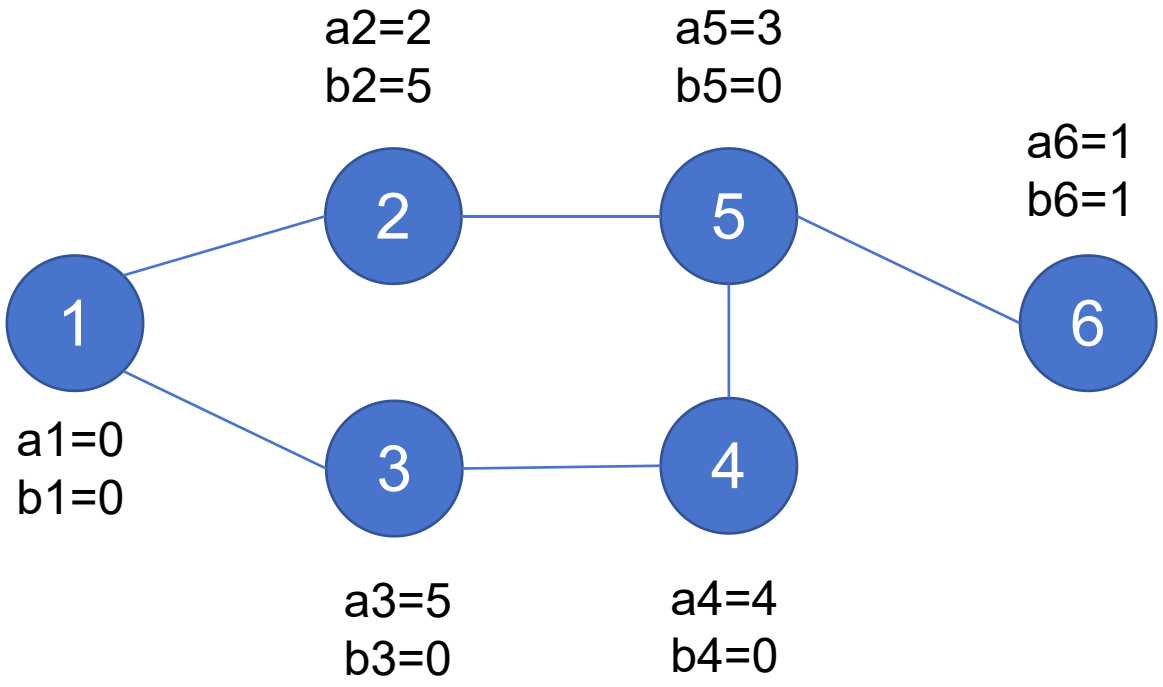


Maintain (c, k) for each u:
c: cost of path 1->u
k: cumulative b from history path 1->u
When we go to v from u, then
push v's states (v, v + a[v] + k, k + b[v]) to pq

Try Modified Dijkstra: Sample 1

states	1	2	3	4	5	6
iter 12	~~(0, 0)~~ ~~(2, 5)~~ ~~(5,0)~~ (...,..) (...,..)	(∞, 0) ~~(2, 5)~~ (9,10) ~~(7,5)~~ (15,10) (...,...)	(∞, 0) ~~(5, 0)~~ ~~(12,5)~~ (17,5)	(∞, 0) ~~(9, 0)~~ (...,...)	(∞, 0) ~~(8, 5)~~ (15,5) ~~(12,0)~~	(∞, 0) (14,6) ~~(13,1)~~

first pop states of node 6
13 is the optimal



Conclusion:

- **can find the optimal**
- **but State Explosion**

Why State Explosion

- A node can be reached via multiple distinct paths
- A node can be reached via a node expanded by itself, with a new accumulated expenses k (MAIN REASON)
- Some paths are clearly worse than others and are not worth expanding.
- Cycles generate many redundant paths.
- Leads to inefficiency

Prevent State Explosion: Dominance Pruning

If one state is worse in both cost and penalty → discard

For two states at same node u: (k1,c1) and (k2,c2)

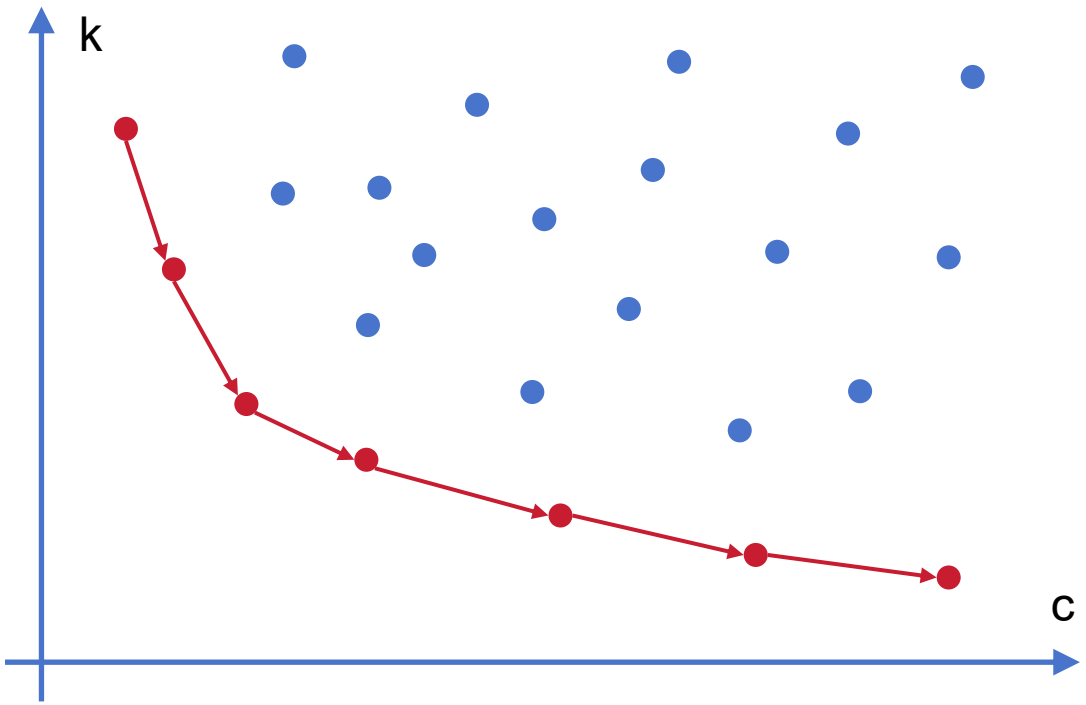
if (k1 <= k2) && (c1 < c2) state 2 is worse, discard.

Implementation Trick:

- Maintain **min_k[u]**, skip states with larger k when popping/pushing
- Traditional shortest path algorithms track distance or visited states per node. With min_k[u] pruning, we only store one value per node instead of all (u,k) states, avoiding memory overflow (MLE)

Why Maintain min_k[u] can work ?

- For all possible states of a node, the batch of states that are most worth preserving must follow the balance between k and c: **either choose a higher c for a lower k, or choose a higher k for a lower c, rather than both being high.**



- Therefore, among these states worth preserving, **as c increases, k must monotonically decrease.**

- Since Dijkstra's algorithm always determines the **smaller states** of c first, **these optimal states will also be determined in descending order of k.**

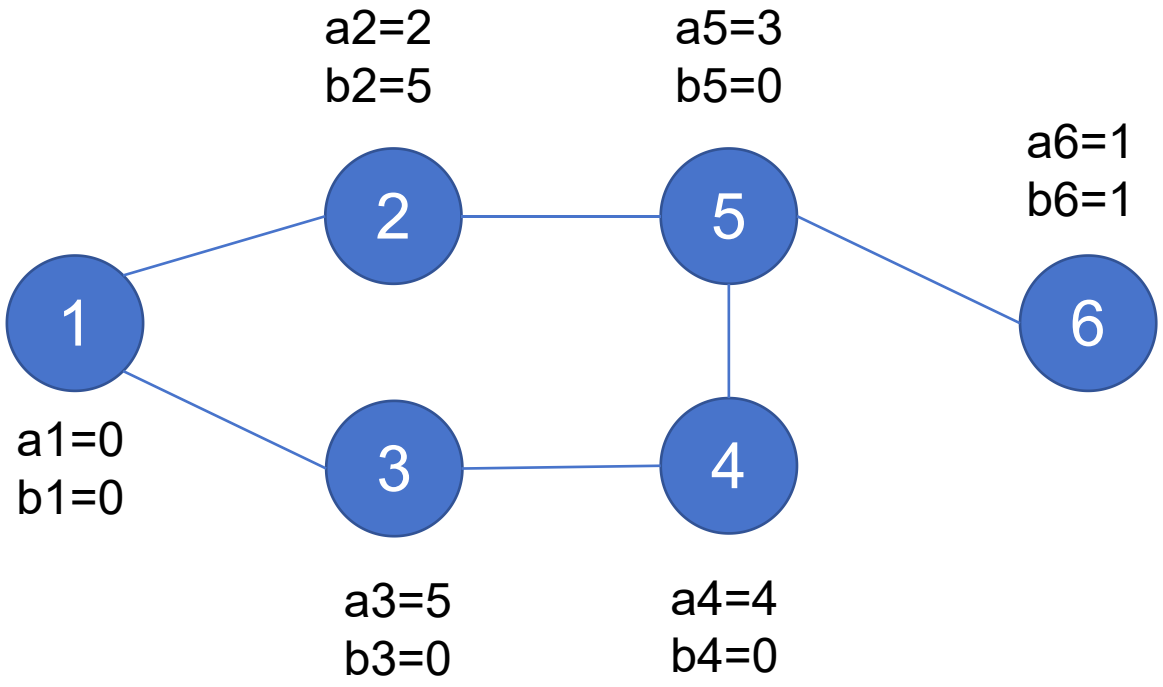
Algorithm Pseudocode

```
min_k[1..n] = INF
PQ.push( (1, 0, 0) )  // (node, c, k)(PQ pop the state with the smallest c first)
min_k[1] = 0

while PQ not empty:
  (u, c, k) = PQ.pop()
  if u == n: return c   // first pop = optimal
  if k >= min_k[u]: continue  // prune dominated, this path no need expand
  min_k[u] = k
  for each neighbor v of u:
    newk = k + b[v]
    newc = c + a[v] + k
    if newk < min_k[v]:   // early pruning
      min_k[v] = newk
      PQ.push( (v, newc, newk) )
```

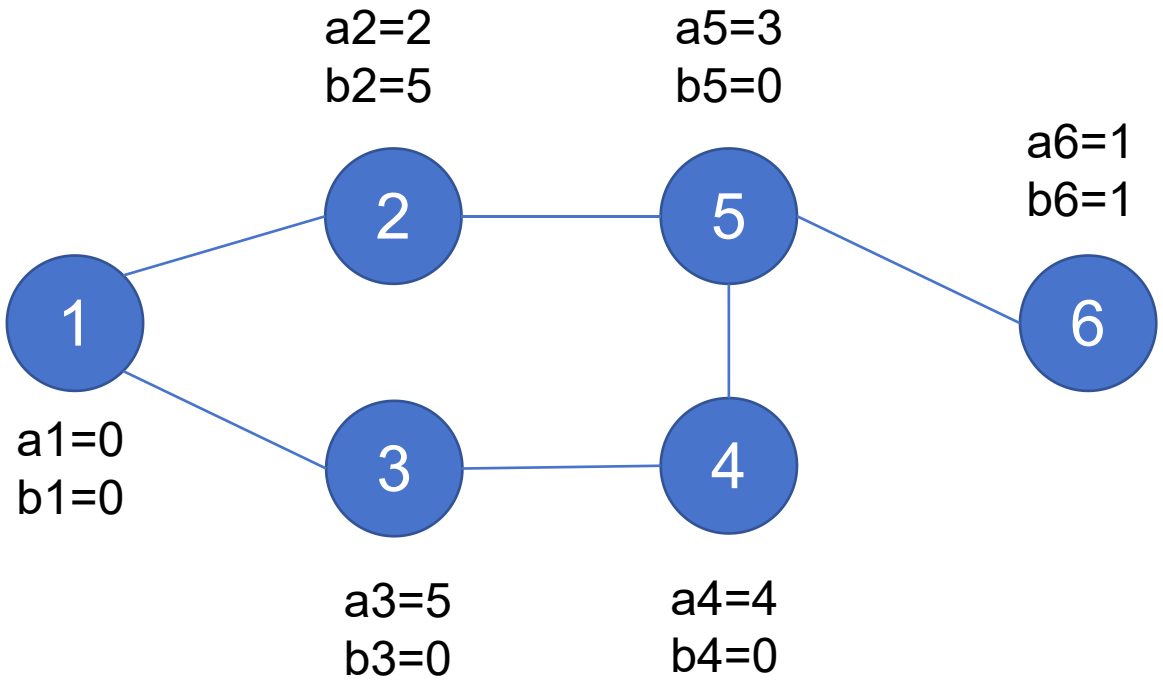
Try Pruned Dijkstra: Sample 1

states	1	2	3	4	5	6
initial	(0, 0)	(∞, 0)	(∞, 0)	(∞, 0)	(∞, 0)	(∞, 0)
iter 1	~~(0, 0)~~	(2, 5) min_k [2] =5	(5, 0) min_k [3] =0	(∞, 0)	(∞, 0)	(∞, 0)
iter 2	~~(0, 0)~~ min_k [0] =0 discard (2,5)	~~(2, 5)~~	(5, 0)	(∞, 0)	(∞, 0) (8, 5) min_k [5] =5	(∞, 0)
iter 3	~~(0, 0)~~ min_k [0] =0 discard (5,0)	~~(2, 5)~~	~~(5, 0)~~	(∞, 0) (9, 0) min_k [4] =0	(∞, 0) (8, 5)	(∞, 0)



Try Pruned Dijkstra: Sample 1

states	1	2	3	4	5	6
iter 4	~~(0, 0)~~	~~(2, 5)~~	~~(5, 0)~~	(∞, 0) (9, 0)	~~(8, 5)~~	(∞, 0) (14, 6) min_k [6] =6
iter 5	~~(0, 0)~~	~~(2, 5)~~	~~(5, 0)~~ min_k [3] =0 discard (12,5)	(∞, 0) ~~(9, 0)~~	~~(8, 5)~~ (12,0)	(14, 6)
iter 6	~~(0, 0)~~	~~(2, 5)~~ min_k [2] =5 discard (12,5)	~~(5, 0)~~	~~(9, 0)~~	~~(8, 5)~~ ~~(12,0)~~	(14, 6) (13, 1)
iter 7	~~(0, 0)~~	~~(2, 5)~~	~~(5, 0)~~	~~(9, 0)~~	~~(8, 5)~~ ~~(12,0)~~	(14, 6) ~~(13, 1)~~



first pop states of node 6
13 is the optimal

Summary

- Apply Dijkstra using extended states
- Apply pruning to improve efficiency